

River Water-Quality Node

How the code works - an overview for COMP1511 students

1. The big picture

The ESP32 is a tiny computer sitting in the river. Every **2 seconds** it reads four sensors: **pH**, **TDS** (dissolved salts), **turbidity** (cloudiness) and **water temperature**. Every **minute** it asks:

- **Is a sensor broken?** For example the reading is impossible, or it hasn't changed in an hour.
- **What's "normal" for this spot?** It keeps a running history to work this out.
- **Does today look different from normal in a way that matches a known pollution pattern?** For example, very acidic water with lots of dissolved stuff looks like heavy metals.
- **What are the other nodes seeing?** This helps work out where the pollution is coming from.

It then prints the result to the screen.

2. One minute of the program, step by step

This is what happens, following `main.cpp`:

```
loop() runs forever (like while (1) { ... })
|- every 2 s: read sensors -> store the sample
'- every 60 s:
    1. take the median of the ~30 samples      (ignores one weird spike)
    2. check for faults                        rules.cpp
    3. compare against "normal"               baseline.cpp
    4. run the pattern rules                   rules.cpp
    5. only alert if it's been true 3 min      persistence.cpp
    6. save this node's result in the array    network.cpp
    7. compare with other nodes               network.cpp
    8. print it all                           display.cpp
```

`setup()` runs once at power-on and `loop()` runs forever. That's how Arduino works instead of `main()`.

3. What each file does

A `.h` file is the **list of what exists** (struct definitions and function prototypes). A `.cpp` file is the **actual code**. This is the same idea as COMP1511 when you split a program into something `.h` and something `.c`.

File	In plain terms
<code>config.h</code>	Every number the program uses: pins, thresholds like "pH < 6.0", and on/off switches. Change settings here, not in the code.
<code>types.h</code>	The structs and enums everything shares. Reading is 4 sensor values. <code>NodeReport</code> is one node's result for a minute.
<code>labels.h</code>	A lookup table (an array of structs) holding the text for each pollution type: its name, likely pollutants and meaning.
<code>sensors.cpp</code>	Talks to the real hardware and reads voltages.
<code>compensation.cpp</code>	Maths that turns voltages into real units (pH, mg/L, NTU) and corrects TDS for temperature.
<code>baseline.cpp</code>	Remembers recent readings in a circular array and works out "normal" as their median.
<code>rules.cpp</code>	The detective. Functions like <code>ruleHeavyMetals()</code> check conditions and return a label plus a confidence.

File	In plain terms
persistence.cpp	Stops false alarms. A rule must be true 3 minutes in a row to turn on, and false 5 minutes in a row to turn off.
node_engine.cpp	Runs steps 1-5 above for one node, using the files above.
network.cpp	The array of nodes along the river, and the code that compares them.
sim.cpp	Fake data (normal, then rain, then an acid spike) so you can demo without real sensors or a river.
display.cpp	All the printing. No other file prints anything.
commands.cpp	Reads commands you type, like add N4 1500.
test/	Tests that run on your laptop, similar to autotests.

4. The node array

This is plain COMP1511 array work. In `network.h`:

```
RiverNode nodes[MAX_NODES]; // array of structs, max 16
uint8_t count_;             // how many are actually used
```

The array is **always kept sorted by distance down the river**:

```
index:      0          1          2          3
node:      N0         N1         N2         N3
distance:  0 m       1000 m     1000 m     2500 m
           '--- same position ---'
```

- **Adding** (`addNode`): find the right spot, shift everything after it one place to the right, then put the new node in. It's the same "insert into a sorted array" exercise you've probably done.
- **Deleting** (`removeNode`): shift everything after it one place to the left.
- **Moving** (`divertNode`): copy the node, delete it, change its distance, then add it again.
- **Two nodes at the same position**: nodes with the same distanceM are side by side, for example opposite banks. No special structure is needed.

Because the array is sorted, "the node upstream of me" is just a smaller index. `assess()` walks backwards through the array to find it.

5. C++ things that aren't in COMP1511 C

Arduino uses C++, so you'll see a few new things. Each has a C equivalent:

You'll see	What it means in C terms
<code>class Baseline { ... };</code>	A struct that also contains its own functions. <code>baseline_.addMinute(...)</code> is like calling <code>baseline_add_minute(&baseline, ...)</code> .
<code>private: / public:</code>	Private fields can only be used by that struct's own functions, which protects them from mistakes elsewhere.
<code>const float X = 6.0f;</code>	Like <code>#define X 6.0</code> , but with a type.
<code>float& mn</code>	Like passing a pointer <code>float *mn</code> , but you write <code>mn = 5</code> instead of <code>*mn = 5</code> .
<code>bool, true, false</code>	Built in, so there's no need for <code>#include <stdbool.h></code> .
<code>uint8_t, uint32_t</code>	Integers with an exact size (8-bit, 32-bit). On a microcontroller, memory matters.
<code>namespace { ... }</code>	Like marking functions <code>static</code> : they're private to that file.

You'll see	What it means in C terms
<code>#pragma once</code>	Stops a header being included twice. It does the same job as <code>#ifndef</code> include guards.
<code>Serial.printf(...)</code>	<code>printf</code> , but sent over the USB cable to your laptop.

6. A suggested reading order

1. `config.h`, to see what settings exist.
2. `types.h`, to learn the structs.
3. `main.cpp`, which is short and shows the whole flow.
4. `rules.cpp`: `ruleHeavyMetals` is a good one to start with, since it's basically a few `if` conditions.
5. `network.cpp`: `addNode` and `removeNode` for the array logic.

Skip `sensors.cpp` and `display.cpp` at first. They're mostly hardware details and formatting.

Try it: to see it all work without any hardware, keep `SIMULATE 1` in `config.h`, upload it, and open the serial monitor. Roughly 4 minutes after start-up you'll see rain get filtered out, then an acid spike trigger an alert.